Interface internals

Contents

What’s actually happening when you write `var r io.Reader = &File{}`. The two-word interface header, the itab, the type assertion mechanism, and the cost of dynamic dispatch.

1. Two interface representations

Go has two runtime structs for interface values, picked at compile time based on whether the interface has methods.

eface: the empty interface

For `interface{}` or `any` (no methods required), the runtime uses `eface`:

```
// runtime/runtime2.go (simplified)
type eface struct {
    _type *_type          // type descriptor
    data  unsafe.Pointer  // pointer to the actual value (or the value itself, if it
        fits)
}
```

Two words. The first identifies the dynamic type. The second points to the value.

iface: the non-empty interface

For any interface with methods (`io.Reader` , `error` , etc.), the runtime uses `iface` :

```
type iface struct {
    tab  *itab           // interface table (type + methods)
    data unsafe.Pointer  // pointer to the actual value
}
```

Two words. The first is the itab, which combines the interface type, the concrete type, and a flat list of method pointers. The second is the value.

Both representations are exactly 16 bytes on 64-bit.

2. The itab structure

```
type itab struct {
    inter *interfacetype  // the interface being satisfied
    _type *_type          // the concrete type
    hash  uint32          // copy of _type.hash for fast type-switch
    _     [4]byte
    fun   [1]uintptr      // variable-length array of method function pointers
}
```

`fun` is the method table: one function pointer per method declared by the interface, in declaration order. A method call through the interface is a lookup at a fixed offset in this table, followed by an indirect call.

3. When data is the value vs a pointer to the value

The second word (`data`) is always a pointer. But:

- If the concrete type is a pointer (`*T`), `data` holds that pointer directly.
- If the concrete type is a value type, the runtime boxes the value onto the heap and `data` points to the box.

```
var i any = 42       // 42 is boxed; data points to a heap int
var p any = &User{}  // already a pointer; data IS the *User
```

This is why putting a `int` into an `any` causes an allocation. Even `bool`, `byte`, and small structs get boxed.

Compiler optimizations

The compiler has special cases for small values that fit in a single word and don't contain pointers:

- Some integer types are stored directly in `data` (without heap allocation) for specific runtime fast paths.
- For most user code, the boxing happens.

Profile with `go build -gcflags=-m` and look for escapes to heap on interface conversions.

4. itab construction and caching

When the compiler sees `var r io.Reader = &File{}`, it has the interface type (`io.Reader`) and the concrete type (`*File`) at compile time. It generates a call to `runtime.getitab(*io.Reader, *File, false)` if it doesn't already have one.

The runtime maintains a global itab cache keyed by (interface type, concrete type). The first lookup verifies that the concrete type implements all methods, then builds and caches the itab. Subsequent lookups are O(1).

If the concrete type does not implement the interface, the conversion fails: panic in non-comma-ok form, `false` in comma-ok form.

For static conversions (where both types are known at compile time and the conversion is guaranteed to succeed), the itab can be a compile-time-known pointer with no runtime lookup.

5. Method dispatch cost

A direct method call:

```
var f *File
f.Read(buf)
// compile target: CALL "".(*File).Read
```

is a single direct call instruction.

An interface method call:

```
var r io.Reader
r.Read(buf)
// compile target:
//   load r.tab
//   load fun[Read offset] from tab
//   CALL through pointer
```

is two loads and an indirect call. The indirect call cost matters because:

- The branch predictor must learn the call target.
- The CPU can't inline through it.
- Modern Spectre mitigations (retpoline on some platforms) add cost per indirect call.

Concretely, an interface call is 1-5 ns more than a direct call, depending on cache state. Inlinable direct calls become free; interface calls never inline.

For most code, this is irrelevant. For inner loops in hot paths processing millions of operations per second, it can dominate.

6. Type assertions

`v, ok := i.(T)` and `v := i.(T)` have different runtime behavior.

Comma-ok form

```
v, ok := i.(T)
```

1. Read `i.tab` (or `i._type` for eface).
2. Compare with the expected `T`.
3. If match: copy `i.data` (or deref if `T` is a value type) into `v`, set `ok = true`.
4. If no match: zero `v`, set `ok = false`.

No panic. Cost: one or two pointer compares, plus the copy.

Single-value form

```
v := i.(T)
```

Same comparison. On mismatch, calls `runtime.panicdottypeI` or similar, which panics with a descriptive message.

Use comma-ok unless you've established the type by other means (e.g., earlier in the same block).

Asserting to an interface

```
r, ok := i.(io.Reader)
```

This is more expensive: the runtime has to check whether the concrete type behind `i` implements `io.Reader`. It calls `runtime.assertI2I` or `runtime.assertE2I`, which:

1. Does an itab cache lookup for (`io.Reader`, dynamic type of `i`).
2. If found, returns it.
3. If not found, verifies method compatibility and caches.

After the first call for a given concrete type, subsequent asserts are cheap.

7. Type switch

```
switch v := i.(type) {
case int:    use(v)
case string: use(v)
case Foo:    use(v)
default:     ...
}
```

The compiler emits a series of type comparisons. For long type switches, the runtime uses a hash-based lookup (similar to a Go map) keyed on `_type.hash`, falling back to linear comparison on collision.

The first case checked is the most common in benchmark dispatch hot paths. Order cases by expected frequency if you care.

8. The nil interface vs interface holding a nil pointer

```
var p *MyError = nil
var err error = p

err == nil      // FALSE
```

The reason is direct from the iface layout:

- A truly nil interface has both `tab = nil` and `data = nil`.
- After `var err error = p`, the interface has `tab = (itab for error/MyError)` and `data = nil`.

The `== nil` check is implemented as “both words are nil.” With a non-nil itab, the check fails even though the value pointer is nil.

This bites callers of functions that return an `error` typed variable:

```
func badReturn() error {
    var e *MyError = nil
    if cond { e = &MyError{} }
    return e        // returns iface{tab: nonNil, data: nil}; caller's nil check fails
}
```

Fix: return concrete `nil`:

```
func goodReturn() error {
    if !cond { return nil }
    return &MyError{}
}
```

9. Method sets and pointer receivers

The compiler determines whether `T` or `*T` satisfies an interface based on the method set.

- `T`'s method set: methods with value receiver `T`.
- `*T`'s method set: methods with receiver `T` or `*T`.

So `*T` always satisfies what `T` satisfies, plus methods declared on `*T`. The opposite isn't true.

```
type Speaker interface { Speak() }
type Dog struct{}
func (d *Dog) Speak() { fmt.Println("woof") }

var s Speaker = &Dog{}        // OK: *Dog has Speak in its method set
var s Speaker = Dog{}         // compile error: Dog does NOT have Speak (only *Dog does)
```

This is a compile-time check. The itab is only constructed for valid (interface, type) pairs.

The auto-address-of for method calls

Note that method calls on a value are different from interface conversion of a value.

```
d := Dog{}
d.Speak()        // OK even though Speak is on *Dog: compiler takes &d automatically
```

This auto-addressing works for direct method calls because `d` is addressable. It doesn't help interface conversion because the interface stores a value, not a reference back to your variable.

10. Interface composition

```
type Reader interface { Read([]byte) (int, error) }
type Writer interface { Write([]byte) (int, error) }
type ReadWriter interface { Reader; Writer }
```

`ReadWriter` is flattened at compile time into a new interface type with both methods. The resulting itab has a method table with `Read` and `Write` slots.

A type satisfies `ReadWriter` if it has both methods. The itab for (`ReadWriter`, `T`) reuses the same compiled method pointers as the individual interfaces would.

11. Empty interface vs typed interface cost

Putting a value into `any`:

```
var i any = 42
```

- Allocates a heap box for the int (8 bytes).
- Writes `_type = *int_type` and `data = ptr_to_box`.
- One allocation, one indirection on read.

Putting a value into a typed interface:

```
var r io.Reader = &File{}
```

- No allocation (the value is already a pointer).
- Writes `tab = *itab_io.Reader_File` and `data = ptr_to_File`.
- The itab lookup is amortized free after first call for this (interface, type) pair.

The cost is roughly:

- Value -> any: ~10 ns + GC pressure.
- Value -> typed interface (if value is already a pointer): ~1-2 ns.
- Method call through interface: ~1-5 ns extra over direct call.

12. How “implements” is checked at compile time

For `var r io.Reader = &File{}`:

1. The compiler reads the method set of `*File`.
2. It checks that every method in `io.Reader` is in that set, with matching signature.

3. If yes, the conversion compiles. The itab will be built (or fetched from cache) at runtime.
4. If no, the conversion is a compile error with a message like:

```
cannot use &File{} (type *File) as type io.Reader:
    *File does not implement io.Reader (missing Read method)
```

Signature matching is exact. `Read(p []byte) (int, error)` and `Read(p []byte) (n int, err error)` are the same signature; named returns don't count. But `Read([]byte) int` doesn't match `Read([]byte) (int, error)`.

13. Interface assertion to "any T" (generics)

Go 1.18 introduced generics. A type parameter constrained to an interface is different from an interface value at runtime.

```
func F[T any](x T) { ... }
```

`x` here is a concrete `T`, not an interface. The compiler generates one or more specializations (via GCShape stenciling) that handle types with similar shapes.

```
func F(x any) { ... }
```

`x` here is an interface. Method calls on `x` go through dynamic dispatch.

Generics avoid the interface overhead for cases where the type set is bounded at compile time. They don't replace interfaces; they complement them.

14. Reflection and interface

`reflect.Value` is roughly:

```
type Value struct {
    typ  *rtype
    ptr  unsafe.Pointer
    flag uintptr
}
```

It's essentially an iface plus extra bookkeeping (settable, kind, etc.). When you call `reflect.ValueOf(x)`, the runtime takes the interface representation of `x` and wraps it.

Reflection ops (Field, Method, Index, Call) are all dynamic and expensive: 50-500 ns per op depending on complexity. Cache `reflect.Type` and `reflect.Method` values when iterating.

15. Common runtime functions

Function	When called
`runtime.convT2E`	concrete value to any (small types, packed)
`runtime.convT2Enoptr`	concrete pointer-free value to any
`runtime.convT2I`	concrete value to typed interface
`runtime.assertE2T` / `assertE2T2`	assert any to type T (with/without comma-ok)
`runtime.assertI2T`	assert typed interface to concrete T
`runtime.assertE2I` / `assertI2I`	assert to another interface
`runtime.getitab`	look up or build itab
`runtime.panicdottype`	failed single-value assertion

These live in `runtime/iface.go`.

16. itab cache and runtime growth

The itab cache is a hash table keyed on (interface type pointer, concrete type pointer). It grows as new (interface, type) pairs are encountered.

For programs that dynamically construct many type combinations (e.g., heavy reflection, plugins, dynamic codegen), the cache can grow significantly. For most programs, it stabilizes at a small size after startup.

The cache is process-global and never shrinks. This is intentional: itabs are tiny and the cache is bounded by the cross-product of interface types and concrete types in the binary.

17. Inspecting interfaces

Print the dynamic type:

```
var i any = 42
fmt.Printf("%T\n", i)              // int

reflect.TypeOf(i)                 // int
reflect.ValueOf(i)                // <int Value>
```

Check if an interface holds a particular type:

```
if _, ok := i.(string); ok { ... }
```

For debugging, `%v` and `%#v` print the value; `%T` prints the type.

18. Performance characteristics

Operation	Cost
Value to any (with boxing)	one heap allocation + 2 word writes
Pointer to typed interface	2 word writes (no allocation)
Method call through interface	direct call cost + 2 loads + 1 indirect call
Type assertion (comma-ok, exact type)	1 pointer compare, plus copy
Type assertion to interface	itab cache lookup (amortized free)
Type switch (N cases)	hashed lookup (effectively constant for many cases)
nil check on interface	2 word compare
Interface equality	type compare + value compare (deep for some types)

19. Best practices

1. Define interfaces on the consumer side. The function that takes the interface declares it.
2. Keep interfaces small. 1-3 methods.
3. Accept interfaces, return concrete types. Caller flexibility, callee precision.
4. Don't use `any` for parameters when generics work. Generics avoid boxing and dynamic dispatch.
5. Be aware of the boxing cost when putting basic types into `any`. Profile if it's in a hot loop.
6. Return concrete `nil` for error, not a typed-nil pointer.
7. Avoid reflect in hot paths. Cache `reflect.Type` if you must.
8. Compile-time interface checks: `var _ I = (*T)(nil)` catches missing methods early.

20. Common gotchas

Gotcha	Cause	Fix
Typed nil != nil	iface holds non-nil tab with nil data	Return concrete nil
Boxing of basic types into any	Allocates one heap object per conversion	Use generics or concrete types
Slow interface method calls in tight loop	Indirect call + no inlining	Use a concrete type for the loop body
Value satisfies T, not interface I	Pointer receiver method	Use `&T{}` or change receiver
Interface assertion panic	Single-value form on wrong type	Use comma-ok
Reflection slow	All ops are dynamic + allocate	Cache Type, avoid in hot paths
Type switch order matters	Cases checked in order	Put common case first

Gotcha	Cause	Fix
Two interfaces with same methods but different names	Distinct types	Convert explicitly

21. Quick reference

Question	Answer
Empty interface struct	`eface { _type, data }` (2 words)
Non-empty interface struct	`iface { tab, data }` (2 words)
itab contents	interface type, concrete type, hash, method func pointers
Where is data	always a pointer; small values boxed onto heap
Method dispatch cost	2 loads + indirect call, no inlining
Type assertion exact-type cost	1-2 pointer compares
Type assertion to interface cost	itab cache lookup (amortized free)
Nil interface	both tab/_type AND data are nil
Typed nil pitfall	non-nil tab + nil data; `== nil` is false
itab cache	global, keyed on (interface, concrete), never shrinks
Method set of T	methods with receiver T
Method set of *T	methods with receiver T or *T
Pointer receiver satisfies	only *T satisfies, not T
any vs generics	any boxes and dispatches; generics specialize
Reflection cost	50-500 ns per op, cache Type
Compile-time interface check	`var _ I = (*T)(nil)`